White Paper

NEXT GENERATION STORAGE TIERING WITH EMC ISILON SMARTPOOLS

Abstract

Most file systems are a thin layer of organization on top of a block device and cannot efficiently address data at large scale. This paper focuses on OneFS, a modern file system that meets the unique needs of Big Data. OneFS includes SmartPools, a native tiering capability that enables enterprises to reduce storage costs without sacrificing performance or data protection.

February 2012

EMC²

Copyright © 2012 EMC Corporation. All Rights Reserved.

EMC believes the information in this publication is accurate as of its publication date. The information is subject to change without notice.

The information in this publication is provided “as is.” EMC Corporation makes no representations or warranties of any kind with respect to the information in this publication, and specifically disclaims implied warranties of merchantability or fitness for a particular purpose.

Use, copying, and distribution of any EMC software described in this publication requires an applicable software license.

For the most up-to-date listing of EMC product names, see EMC Corporation Trademarks on EMC.com.

VMware is a registered trademark or trademark of VMware, Inc. in the United States and/or other jurisdictions. All other trademarks used herein are the property of their respective owners.

Part Number h8321

Table of Contents

Introduction 4

Isilon Scale-Out NAS Architecture 4

- Single File System 5
- Data Layout/Striping 6
- File Writes 6
- Job Engine 8
- Data Protection in OneFS 10
- Failure Domains and Resource Pools 11
- Restriping 11
- SmartCache 12

Smart Pools 13

- Overview 13
- Disk Pools 14
- File Pools 16
- Custom File Attributes (also referred to as User Attributes) 19
- Data Location 20
- Performance with SmartPools 21
- Using SmartPools to Improve Performance 22
- Data Access Settings 22
- Leveraging SSDs for Metadata & Data Acceleration 23
- Minimizing the Performance Impact of Tiered Data 24

SmartPools Use Cases – Lowering Storage Costs without sacrificing performance or protection 25

- Use Case 1: Storage Cost Reduction in Post Production 25
- Use Case 2: Data Protection in Manufacturing Design 26
- Use Case 3: Performance with Global Namespace Acceleration 27
- Use Case 4: Investment Protection for Financial Market Data 28

Conclusion 29

About EMC Isilon 29

Introduction

The value proposition of storage tiering is very strong: mix lower-cost storage with higher performance, storage to lower the overall cost of storage while maintaining the required application performance. The true benefits of storage tiering can only be accomplished if the total cost of ownership is also lowered by providing simplicity, scalability and enterprise capabilities.

EMC Isilon's SmartPools is a next generation approach to tiering. The SmartPools capability is native to the architecture of Isilon OneFS, a scale-out file system, which allows for unprecedented flexibility, granularity, and ease of management. SmartPools is integrated with and depends on many of the structures within the OneFS operating system. To understand how SmartPools works, this paper will first examine the OneFS architecture and underlying capabilities.

Isilon Scale-Out NAS Architecture

Traditional Network Attach Storage (NAS) architecture is based on scale up – shelves of disk storage behind disk controllers. When more capacity is needed, more shelves are added until the upper limit is reached or controller performance slows to an unacceptable level. When this happens, more controllers are added with drives behind each. Storage in the scale up model is segmented into many levels of physical and logical compartments – RAID (LUNs, stripe groups, concatenations, etc.), Volumes, File systems – each of which has to be configured and managed, and most of which add complexity to provisioning and scaling.

In Isilon's scale-out Network Attached Storage, OneFS combines the three layers of traditional storage architectures—file system, volume manager, and RAID—into one unified software layer, creating a single intelligent distributed file system that runs on an Isilon storage cluster.

Figure 1. OneFS combines file system, volume manager and protection into one single intelligent, distributed system.

This is the core innovation that directly enables enterprises to successfully utilize scale-out NAS in their environments today. It adheres to the key principles of scale-out; intelligent software, commodity hardware and distributed architecture. OneFS is not only the operating system, but also the underlying file system that drives and stores data in the Isilon storage cluster.

Single File System

The importance of a single file system that operates outside the constraints of traditional scale up constructs like RAID groups is that data can be placed anywhere on the cluster, while also allowing for different levels of performance and protection.

Each cluster creates a single namespace and file system. The file system is distributed across all nodes in the cluster and is accessible by clients connecting to any node in the cluster. There is no partitioning, and no need for volume creation. Instead of limiting access to free space and to non-authorized files at the physical volume level, OneFS provides for the same functionality in software by using share and file permissions, and Isilon's SmartQuotas service, which provides directory-level quota management.

Because all information is shared among nodes across the internal network, data can be written to or read from any node, thus optimizing performance when multiple users are concurrently reading and writing to the same set of data.

From an application or user perspective, all capacity is available - the storage has been completely virtualized for the users and administrator. The file tree can grow organically without requiring planning or oversight about how the tree grows or how users use it. No special thought has to be applied by the administrator about tiering files to the appropriate disk, because SmartPools will handle that automatically without disrupting the single tree. No special consideration needs to be given to how one might replicate such a large tree, because the Isilon SyncIQ service automatically parallelizes the transfer of the file tree to one or more alternate clusters, without regard to the shape or depth of the file tree.

A single file system approach over a scalable hardware model means Isilon scale-out NAS can deliver what traditional NAS architectures cannot, including:

- 80% of Raw utilization
- Scalability to 15 Petabyte with linearly increasing performance
- Single point of management
- 'No node left behind' investment protection
- Rapid integration of new drives and new technologies
- Higher data protection options
- Granular control of performance and management
- Reduced costs with integrated, transparent tiering

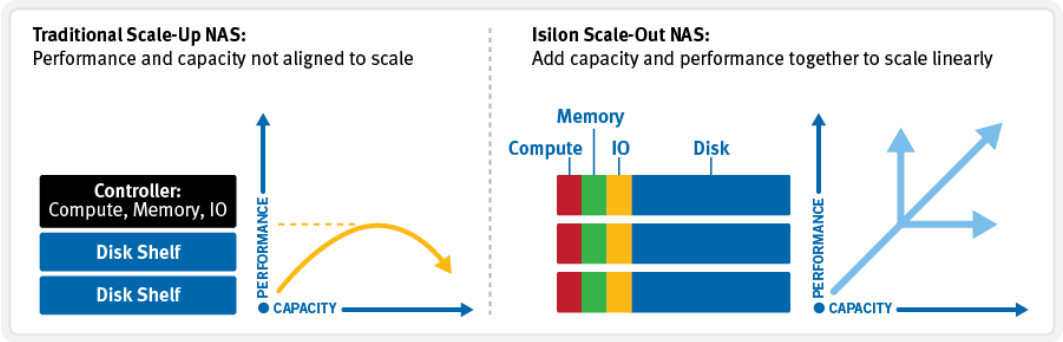

Figure 2. Comparison of Traditional Scale-Up NAS with Isilon Scale-Out NAS

Data Layout/Striping

OneFS uses physical pointers and extents for metadata and stores file and directory metadata in inodes. B-trees are used extensively in the file system, allowing scalability to billions of objects and near-instant lookups of data or metadata. OneFS is a completely symmetric, and highly distributed file system. Data and metadata are always redundant across multiple hardware devices. Data are generally protected using erasure coding, for efficiency reasons, enabling up to 80% on clusters of five nodes or more. Metadata (which makes up generally less than 1% of the system) is mirrored for performance reasons. As OneFS does not rely on RAID for data protection, the amount of redundancy is selectable by the administrator, allowing for up to file-level granularity. Metadata access and locking tasks are managed by all nodes collectively and equally in a peer-to-peer architecture. This symmetry is key to the simplicity and resiliency of the architecture. There is no single metadata server, lock manager or gateway node.

File Writes

When a client connects to a OneFS-managed node and performs a write operation, it is connecting to the top half or initiator of that node, as shown in Figure 3. Files are broken into smaller logical chunks called stripes before being written to the bottom half or participant of a node (disk). Failure-safe buffering using a write coalesce is used to ensure that writes are efficient and read-modify-write operations are avoided. The size of each file chunk is referred to as the stripe unit size.

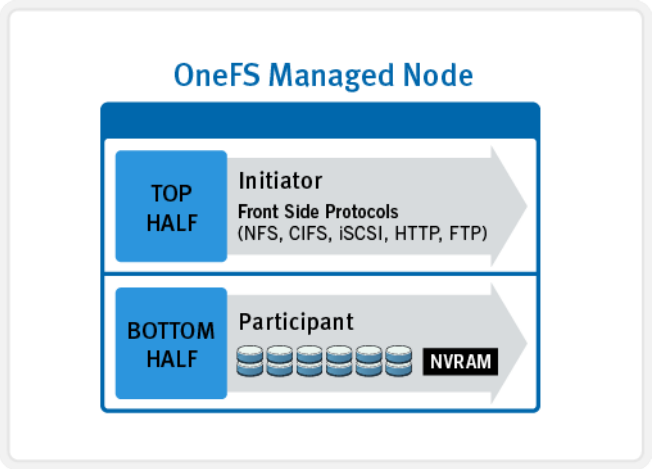

Figure 3. EMC Isilon OneFS Managed Node

OneFS stripes data across all nodes—and not simply across disks—using standard erasure-code or mirroring technology. For data, OneFS can use (at the administrator's discretion) either the Reed-Solomon erasure coding system for data protection, or (less commonly) mirroring. Mirroring, when applied to user data, tends to be used more for high-transaction performance cases, such as iSCSI LUNs. The bulk of user data will generally use erasure coding, as it provides extremely high performance without sacrificing on-disk efficiency. Erasure coding can provide beyond 80% efficiency on raw disk with five nodes or more, and on large clusters can even do so while providing quadruple-level redundancy. The stripe width for any given file is the number of nodes (not disks) that a file is written across. It is determined by the number of nodes in the cluster, the size of the file, and the protection setting (for example, N+2).

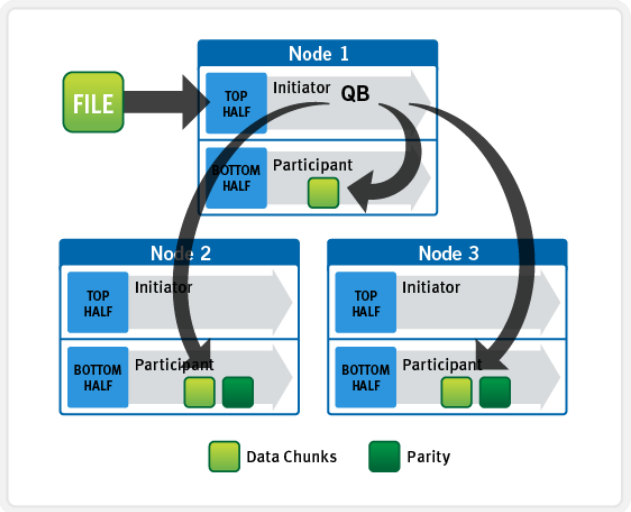

Figure 4. OneFS Distributed Transactions and Two Phase Commit

OneFS uses advanced algorithms to determine data layout for maximum efficiency and performance. When a client connects to a node, that node's initiator acts as the

"quarterback" for the write data layout of that file as depicted in Figure 4. In an Isilon cluster, data, parity, metadata and inodes are all distributed on multiple nodes, and even across multiple drives within nodes. Figure 5 below shows a file write operation across all nodes in a 3 node cluster.

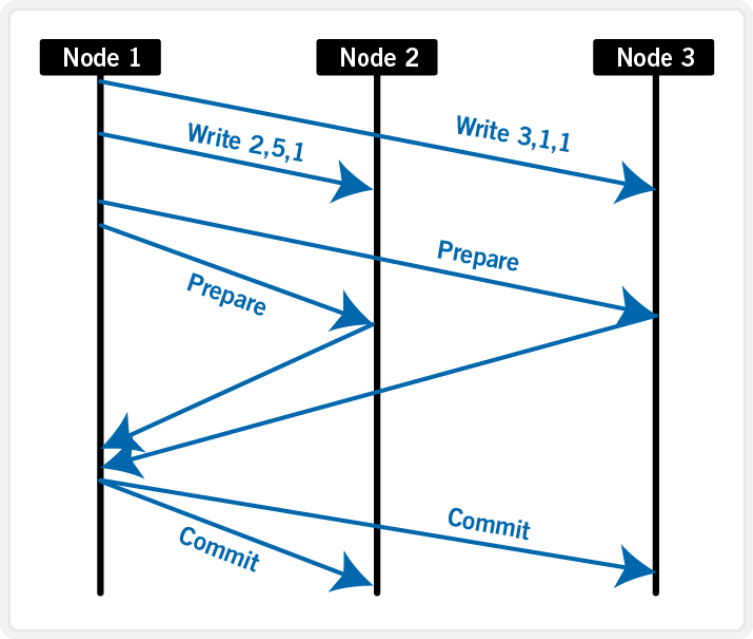

Figure 5. File Write Operation Across a 3 Node Cluster

Every node that owns blocks in a particular write is involved in a two-phase commit. The mechanism relies on NVRAM for journaling all the transactions that are occurring across every node in the storage cluster. Using multiple NVRAMs in parallel allows for high-throughput writes while maintaining data safety against all manner of failures, including power failures. In the event that a node should fail mid-transaction, the transaction is restarted instantly without that node involved. When the node returns, the only required actions are for the node to replay its journal from NVRAM—which takes seconds or minutes—and, occasionally, for AutoBalance to rebalance files that were involved in the transaction. No expensive "fsck" or "disk-check" processes are ever required. No drawn-out resynchronization ever needs to take place. Writes are never blocked due to a failure. EMC Isilon's patented transaction system is one of the ways that OneFS eliminates single—and even multiple—points of failure.

In a write operation, the initiator manages the layout of data and metadata; the creation of erasure codes; and the normal operations of lock management and permissions control.

Job Engine

The Job Engine is at the core of OneFS, and in charge of distributing and performing critical jobs or operations throughout the entire cluster. The Job Engine organizes and manages "jobs" – their priority, timing, resources, execution verification, etc A "job" is defined as a set of one or more phases which effects some logical change on the

cluster. The priority level and impact policy of a job can be set in relation to other jobs. Below is a list of jobs or operations that the Job Engine performs:

AutoBalance	Automatically balances free space in the cluster.
AVScan	Scans files for viruses and malware, in conjunction with a third party ICAP server.
Collect	Reclaims disk space that could not be freed due to node or drive unavailability.
FlexProtect	Re-protects files and directories after a node or device failure.
FSAnalyze	Gathers file system analytics data for use by the InsightIQ application.
MediaScan	Scrubs disks for media-level errors as a periodic background task.
MultiScan	Runs Collect and AutoBalance jobs concurrently to optimize their performance.
QuotaScan	Used by SmartQuotas to account for existing files when a quota is created.
SetProtectPlus	Used to set default system protection settings without SmartPools.
SnapshotDelete	Frees disk space that is associated with deleted snapshots.
TreeDelete	Deletes a path within OneFS in a highly parallelized, multi-node process.
SmartPools	Enforces SmartPools Disk Pools and File Pool Policies

Table 1. Isilon OneFS Job Functions

The OneFS Job Engine handles all the background cluster tasks: creating jobs for each task, prioritizing them and ensuring that inter-node communication, and cluster wide capacity utilization and performance are balanced and optimized. Job Engine ensures that core cluster functions have priority over less important work and give applications integrated with OneFS – Isilon add-on software or applications integrating to OneFS via the OneFS API – the ability to control the priority of their various functions to ensure the best resource utilization.

SmartPools translates the administrators' requirements for different data to be treated in different ways into requests to the OneFS Job Engine. OneFS executes the work dictated by SmartPools configuration options. For example, a data movement requested by SmartPools works just the same as a data movement OneFS would orchestrate to rebuild a failed drive, AutoBalance, or data movement to take advantage of newly added capacity. Data movement within OneFS is always a transparent reallocation of underlying physical blocks; at no time are file system "stubs" or "pointers" used.

The Job Engine organizes and manages Jobs – their priority, timing, resources, execution verification, etc. Each Job can be multiple phases and each phase can contain multiple items where each item is the smallest amount of work that can be

done by the job engine, for example, a LIN[1] to be restriped. Items are assigned to "Tasks" which are then each assigned to a single thread[2].

Each job has an "Impact Profile" which characterizes how much system resources the job will take and an Impact Policy and an "Impact Schedule", both of which are configurable and dictate how the Impact is spread out over resources over time. Essentially, the amount of work a job takes is fixed, but can be configured to meet user requirements to minimize system impact when the work is done.

Critically, jobs also have "Priorities" which dictate order of execution. Core system jobs such as drive rebuilds, for example, would be higher priority than a SmartPools data movement. In general, Job Engine priorities are defaulted to this descending priority order, as described below.

Data Protection in OneFS

OneFS is designed to withstand multiple simultaneous component failures (currently four) while still affording unfettered access to the entire file system and dataset. Data protection is implemented at the file system level and, as such, is not dependent on any hardware RAID controllers. This provides many benefits, including the ability add new data protection schemes as market conditions or hardware attributes and characteristics evolve. Since protection is applied at the file level, a OneFS software upgrade is all that's required to make new protection and performance schemes available.

OneFS employs the popular Reed Solomon erasure coding algorithm for its parity protection calculations. Protection is applied at the file level, enabling the cluster to recover data quickly and efficiently. Inodes, directories and other metadata are protected at the same or higher level as the data blocks they reference. Since all data, metadata and forward error correction (FEC) blocks are striped across multiple nodes, there is no requirement for dedicated parity drives. This guards against both single points of failure and bottlenecks, and allows file reconstruction to be a highly parallelized process. OneFS provides N+1 through N+4 parity protection levels, providing protection against up to four simultaneous component failures. A single failure can be relatively minor, such as an individual disk or, very significant, as in the case of the failure of an entire node.

OneFS also supports several hybrid protection schemes. These include N+2:1 and N+3:1, which protect against two drive failures or one node failure, and three drive failures or one node failure, respectively. These protection schemes are particularly useful for high density node configurations, where each node contains up to thirty six, multi-terabyte SATA drives. Here, the probability of multiple drives failing far surpasses that of an entire node failure. In the unlikely event that multiple devices have simultaneously failed, such that the file is "beyond its protection level," OneFS will re-protect everything possible and report errors on the individual files affected to the cluster's logs.

[1]The LIN tree maps logical inode numbers (64-bit globally unique) to the set of on-disk inode locations associated with the file.
[2] The Isilon cluster Job Engine is moving from single threaded to multi-threaded architecture in 2012

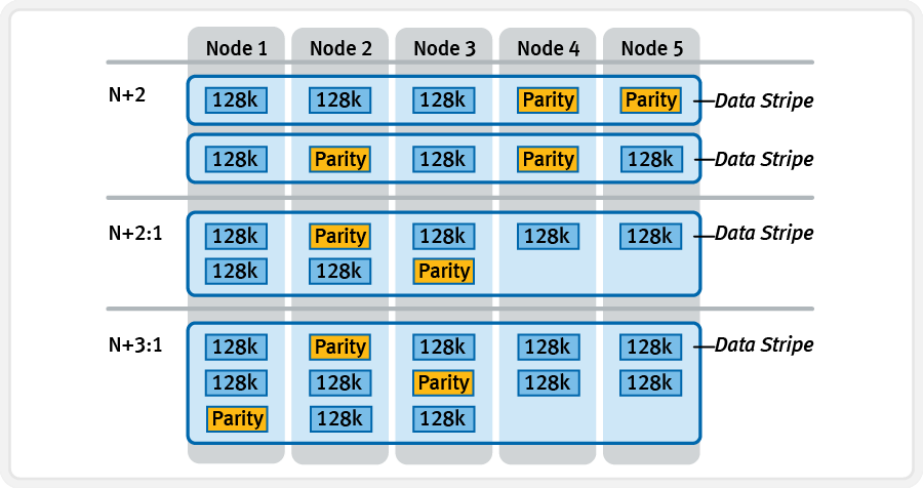

Figure 6. OneFS Hybrid Parity Protection Schemes (N+M:x)

As mentioned above, OneFS also provides a variety of mirroring options ranging from 2x to 8x, allowing from two to eight mirrors of the specified content. Metadata, for example, is mirrored at one level above FEC by default with the exception being directories which are mirrored at two levels above FEC.

Striped, distributed metadata coupled with continuous auto-balancing affords OneFS truly linear performance characteristics, regardless of fullness of file system. Both metadata and file data are spread across the entire cluster keeping the cluster balanced at all times.

Failure Domains and Resource Pools

SmartPools is the data tiering and management framework used by OneFS. For data protection purposes, SmartPools facilitates the subdivision of a large cluster into smaller disk pools without requiring subdivision of the file system. This transparent partitioning increases the average mean time to data loss (MTTDL), an important measurement of file system reliability. For example, an 80-node nearline cluster would typically run at a +4 protection level. However, partitioning it into four, twenty node disk pools would allow each pool to run at +2, thereby lowering the protection overhead and improving data utilization without any net increase in management overhead. SmartPools balances files across these pools, taking advantage of all available capacity and ensuring data movement is completely transparent to applications and end users.

Restriping

The key purpose of the AutoBalance feature of OneFS is to reallocate and rebalance data and make storage space more usable and efficient. In most cases, the stripe width of larger files can be increased to take advantage of new free space (as nodes are added) and to make the on-disk striping more efficient.

OneFS does not rely on hardware RAID either for data allocation, or for reconstruction of data after failures. Instead, OneFS manages protection of file data directly, and

when a failure occurs, it rebuilds data in a parallelized fashion. OneFS is able to determine which files are affected by a failure in constant time, by directly reading inode data linearly from the underlying storage media. The set of affected files are assigned to a set of worker threads that are distributed among the cluster nodes by the job engine. The worker nodes repair the files in parallel. This implies that as cluster size increases, the time to rebuild from failures decreases. This is a highly efficient way to maintain the resiliency of clusters as their size increases.

Most traditional storage systems based on RAID require the provisioning of one or more hot sparedrives to allow independent recovery of failed drives. The hot spare drive replaces the failed drive in a RAID set. If these hot spares are not themselves replaced before more failures appear, the system risks a catastrophic data loss. OneFS avoids the use of hot spare drives, and simply borrows from the available free space in the system in order to recover from failures; this technique is called ***virtual hot spare***. This approach allows the cluster to be fully self-healing, without human intervention. The administrator can create a virtual hot spare reserve, which prevents users from consuming capacity that is reserved for the virtual hot spare.

The process of choosing a new layout is called ***restriping,*** and is illustrated below in Figure 7. The mechanism is identical for repair, rebalance, and tiering. Data is moved in the background with the file available at all times, completely transparent to end users and applications.

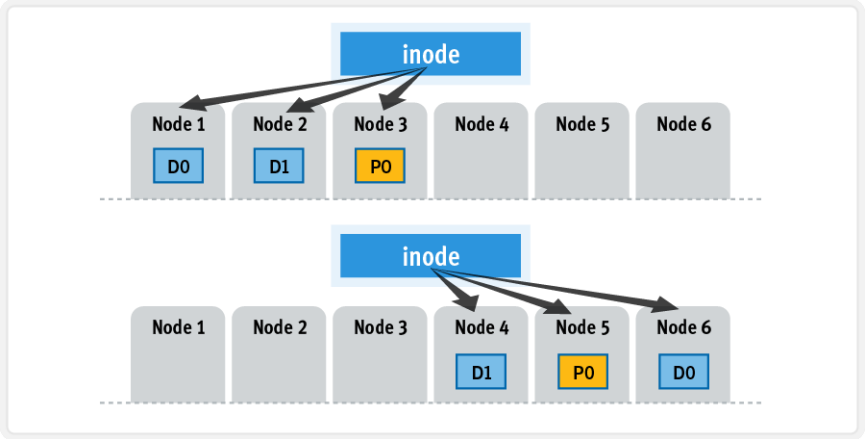

Figure 7. Restriping Example

SmartCache

OneFS SmartCache is a globally coherent read and write cache that provides faster access to content. Like other resources in the cluster, as more nodes are added, the total cluster cache grows in size, enabling Isilon to deliver unparalleled performance while maintaining globally coherent read and write access across the cluster. The shared cache is used to pre-fetch both metadata and file data to optimize access based on the actual workflows in effect.

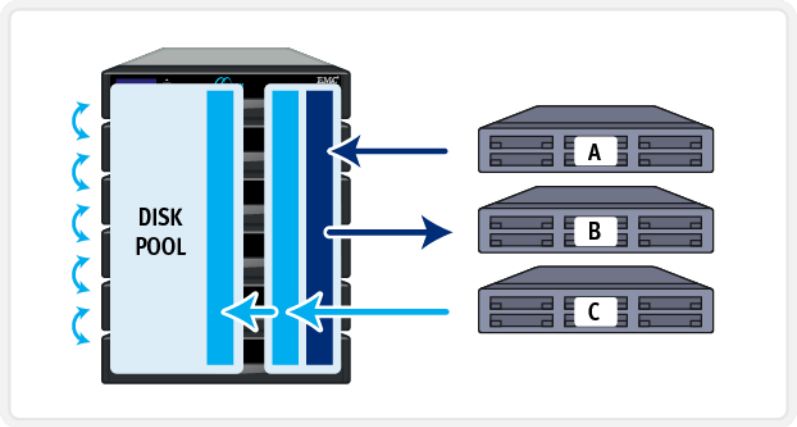

Figure 8. OneFS SmartCache Global Cache Coherency

Write caching uses write buffering to accumulate, or coalesce, multiple write operations to NVRAM so that they can be written in to disk more efficiently without risking data integrity. This form of buffering reduces the disk write penalty which requires multiple reads and writes for each write operation. Write caching maintains global cache coherency, allowing a read-after-write to occur across nodes in the cluster. New data writes are available for reading by all nodes within the cluster, even before data is flushed to disk. For example, as illustrated in Figure 8, as server C is writing data, other clients, such as server B, have access to that data from cache. The cache algorithms then flush the cache to disk as shown when server C stops writing data, or periodically to streamline performance. Also, when server A writes to the same file, SmartCache ensures any writes to the same file from server C, that are still in cache, are first flushed to disk.

Smart Pools

Overview

SmartPools builds on the core OneFS functionality described above including the OneFS single file system, data layout and striping, Job Engine and SmartCache - and adds two additional key concepts: Disk Pools and File Pools.

Disk Pools are groups of similar Isilon storage nodes. Multiple groups of different node types can work together in a single cluster – for example: one Disk Pool of S Series nodes typically used for IOPS-intensive applications, one Disk Pool of X Series nodes typically used for high-concurrent and sequential workloads and one Disk Pool of NL Series nodes typically used for near-primary accessibility with near-tape value. This means different node types of different performance and capacity characteristics can be mixed to meet different workflow requirements with the same single point of management. It also means older and newer hardware can be mixed allowing for simple, significant investment protection even across product generations.

File Pools are user configurable policies which dictate how data is placed, protected, accessed and how it moves among the Disk Pools. This means data can be

automatically moved from one type of storage to another within a single cluster to meet performance, space, cost or other requirements, while retaining its data protection settings. For example a File Pool policy may dictate anything written to path /ifs/foo goes to the S Series nodes in Disk Pool 1, then moves to the NL Series nodes in Disk Pool 3 when older than 30 days.

To simplify management, there are defaults in place for Disk Pool and File Pools settings which handle basic data placement, movement, protection and performance. All of these can also be configured via the simple and intuitive UI, delivering deep granularity of control.

When a SmartPools job runs, any data which needs to be changed to meet a File Pool policy is changed by the file system itself – moved, re-laid out, etc. There are no stubs. The file system itself is doing the work so no transparency or file system alignment technical or data risks apply.

Data movement is parallelized with the resources of multiple nodes being leveraged for speedy job completion. While a job is in progress all data is completely available to users and applications.

The performance of different nodes can also be augmented with the addition of system cache or Solid State Drives (SSDs). An Isilon scale-out NAS cluster can leverage over 14 TB of globally coherent cache. Most nodes can be augmented with up to twenty-four SSD drives.

Within a File Pool SSD 'Strategies' can be configured to place a copy of that pool's metadata, or even some of its data on SSDs in that pool. When Global Namespace Acceleration (GNA) is enabled, a copy of all metadata for the entire cluster is kept on SSD, so access to all data on the cluster – even data on nodes which have no SSDs in them – is accelerated. For data that never needs acceleration, the correct SSD strategy is to avoid SSDs.

Overall system performance impact can be configured to suit the peaks of and environments workload. Change the time or frequency of any SmartPools job and the amount of resources allocated to SmartPools. For extremely high-utilization environments, a sample File Pool policy can be used to match SmartPools run times to non-peak computing hours. While resources required to execute SmartPools jobs are low and the defaults work for the vast majority of environments, that extra control can be beneficial when system resources are heavily utilized.

Disk Pools, File Pool Policies, SmartPools Jobs, data placement and performance are all discussed in detail below.

Disk Pools

A Disk Pool is a logical grouping of disks across multiple nodes within a cluster. Each Disk Pool contains disk only in the same type of storage nodes. For example, S Series nodes with 300 GB SAS drives and 1,200 GB SSD per node would be in one pool, whereas NL Series with 3 TB SATA Drives would be in another.

Today, the smallest increment in a Disk Pool is a node and a minimum of 3 nodes are required per pool.

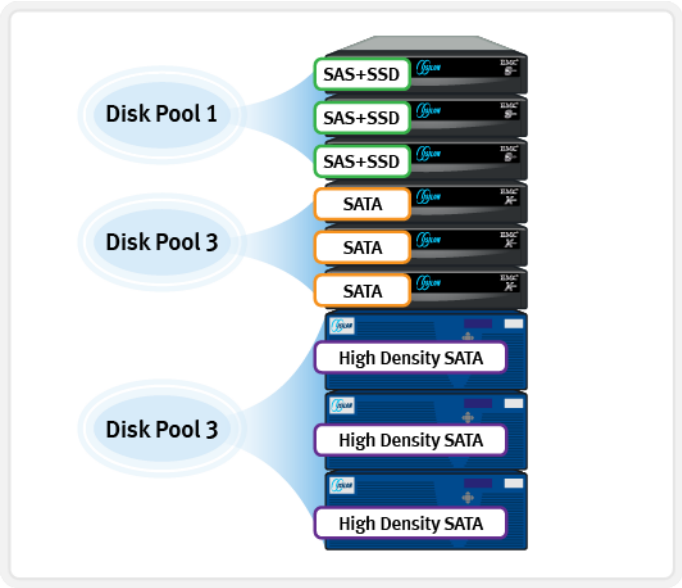

Figure 9. Disk Pool Example

SmartPools users typically deploy 2 to 4 pools with the fastest pool containing nodes with some or all SSDs for the fastest, most demanding portions of a workflow, and the slowest containing multi-TB SATA drive nodes.

Once pools are created, they can be easily modified to adapt to changing requirements. Individual nodes can be reassigned from one pool to another. One Disk Pool can be assigned to another Disk Pool effectively merging two pools together. Disk Pool set up can be discarded, dissolving Pool configurations and releasing member nodes so they can be added to new or other existing pools. Changes to Disk Pool memberships are simple to do and can be made effective immediately. Disk Pools can also be renamed at any time without changing any other settings in the Disk Pool configuration.

Any new node added to a cluster is automatically allocated to a Disk Pool[3] without any additional configuration steps, inheriting the SmartPools configuration properties of that Disk Pool. This means the configuration of Disk Pool data protection, and other settings only needs to be completed once per Pool and can be done at the time the Pool is first created. Automatic allocation is determined by the shared attributes of the new nodes with the closest matching Disk Pool (an S node with 600 GB SAS Drives joins a Disk Pool of S Nodes with 600 GB drives). If the new node is not a close match to the nodes of any existing Disk Pool it remains un-provisioned until the minimum Disk Pool node membership for like nodes is met (3 nodes of same or similar storage and memory configuration).

[3] From OneFS 6.5.3 forward

When a new Disk Pool is created and nodes are added SmartPools associates those nodes with an ID. That ID is also used in File Pool policies and file attributes to dictate file placement in a specific pool.

Disk Pool ID	Nodes
1	1,2,3
2	4,5,6,7,8,9,
…	…

By default, a file which is not covered by a File Pool policy specifically assigning it to a Disk Pool will go to the default Disk Pool or pools identified during set up. If no default is specified, SmartPools will write that data to the pool with the most available capacity.

In practice, default policies are always used because they can be very powerful. Most administrators do not want to set rules to govern all their data. They are generally concerned about some or most of their data in terms of where it sits and how accessible it is, but there is always data for which location in the cluster is going to be less important. For this data, there is the default policy, which is used for files for which none of the other policies in the list have applied. Typically, it is set to optimize cost and to avoid using storage pools that are specifically needed for other data. For example, most default policies are at a lower protection level, and use only the least expensive tier of storage.

If a Disk Pool fills up, writes to that pool will automatically spillover to the next pool. This default behavior ensures that work can continue even if one type of capacity is full. There are some circumstances in which spillover is undesirable, such as when different groups own different pools, data location creates concerns for security, and so on. In these circumstances, spillover can simply be turned off. Turning spillover off ensures a file exists in one pool and will not move to another[4]. Keep in mind that reservations for virtual hot sparing[5] will effect spillover – if, for example, VHS is configured to reserve 10% of a pool's capacity, spillover will occur at 90% full.

Protection settings can be configured outside SmartPools and managed at the cluster level, or within SmartPools at the Disk Pool or File Pool level. Wherever protection levels exist, they are fully configurable. The default protection setting in a Disk Pool is 2:1[6].

File Pools

File Pool policies are used to dictate how files move among Disk Pools, their layout, access patterns and protections settings. A File Pool policy is built on a file attribute the policy can match on. The attributes a file Pool policy can use are any of: File

[4] There are some exceptions - See 'Data Location' below
[5] See 'Restriping' section above for details on virtual hot sparing
[6] See 'Data Protection in OneFS' above

Name, Path, File Type, File Size, Modified Time, Create Time, Metadata Change Time, Access Time or User Attributes.

Once the file attribute is set to select the appropriate files, the action to be taken on those files can be added – for example: if the attribute is File Size, additional settings are available to dictate thresholds (all files bigger than… smaller than…). Next actions are applied: move to Disk Pool x, set to y protection level and lay out for z access setting.

File Attribute	Description
File Name	Specifies file criteria based on the file name
Path	Specifies file criteria based on where the file is stored
File Type	Specifies file criteria based on the file-system object type
File Size	Specifies file criteria based on the file size
Modified Time	Specifies file criteria based on when the file was last modified
Create Time	Specifies file criteria based on when the file was created
Metadata Change Time	Specifies file criteria based on when the file metadata was last modified
Access Time	Specifies file criteria based on when the file was last accessed
User Attributes	Specifies file criteria based on ***custom*** attributes – see below

Table 2. OneFS File Pool File Attributes

'And' and 'Or' operators allow for the combination of criteria within a single policy for extremely granular data manipulation.

FFile Pool Policies that dictate placement of data based on its path force data to the correct disk on write directly to that Disk Pool without a SmartPools job running. File Pool Policies that dictate placement of data on other attributes besides path name get written to Disk Pool with the highest available capacity and then moved, if necessary to match a File Pool policy, when the next SmartPools job runs. This ensures that write performance is not sacrificed for initial data placement.

As mentioned above in the Disk Pools section, any data not covered by a File Pool policy is moved to a tier that can be selected as a default for exactly this purpose. If no Disk Pool has been selected for this purpose, SmartPools will default to the Disk Pool with the most available capacity

When a File Pool policy is created, SmartPools stores it in the Isilon OneFS configuration database with any other SmartPools policies. When a SmartPools job runs, it runs all the policies in order. If a file matches multiple policies, SmartPools will apply only the first rule it fits. So, for example if there is a rule that moves all jpg files to a nearline Disk Pool, and another that moves all files under 2 MB to a performance tier, if the jpg rule appears first in the list, then jpg files under 2 MB will go to nearline, NOT the performance tier. As mentioned above, criteria can be

combined within a single policy using 'And' or 'Or' so that data can be classified very granularly. Using this example, if the desired behavior is to have all jpg files over 2 MB to be moved to nearline, the File Pool policy can be simply constructed with an 'And' operator to cover precisely that condition.

Policy order, and policies themselves, can be easily changed at any time. Specifically, policies can be added, deleted, edited, copied and re-ordered.

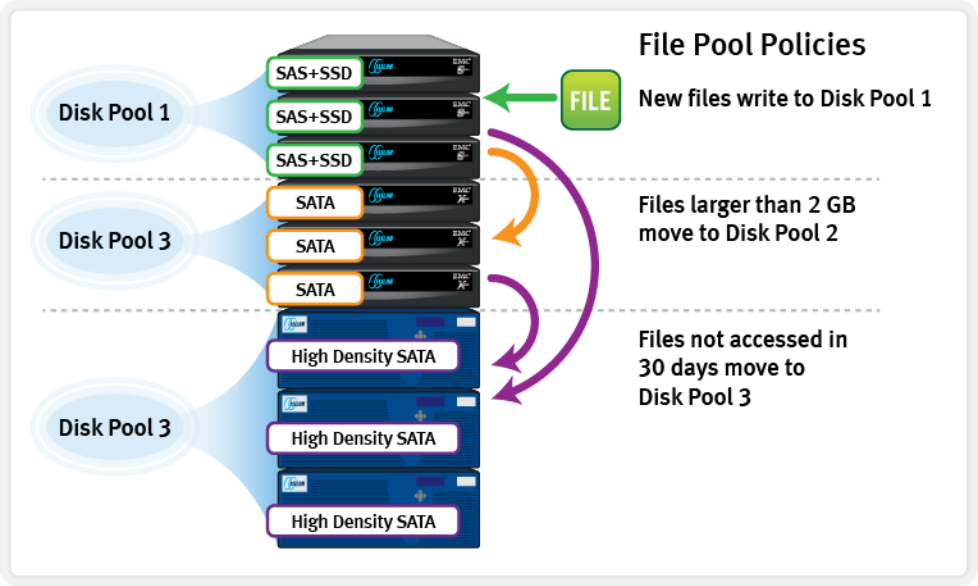

Figure 10. File Pool Policy Example

Figure 10 illustrates a common case – the user is working with time sensitive data and wants to reserve the fastest Disk Pool (SAS + SSD) for the freshest data. They want to optimize their performance by favoring small files on the faster systems also, moving even relatively new data that is larger in size to their second tier Disk Pool. Lastly, they want to cost reduce older data as soon as possible so there is a default policy to move any data not accessed in ‹30 days to the least expensive Disk Pool.

As the list of File Pool policies grows (SmartPools currently supports up to 128 policies), it becomes less practical to manually walk through all of them to see how a file will behave when policies are applied. SmartPools also has some advanced options which are available on the command line for this kind of scenario testing and trouble shooting[7].

File Pool policies can be created, copied, modified, prioritized or removed at any time. Sample policies have been provided as references that can be used as is or modified, however most users with even a little experience report it is easier to just build their own from scratch as the process in the UI typically takes less than one minute per policy.

[7] See 'Anatomy of a SmartPools Job' below

Custom File Attributes (also referred to as User Attributes)

Custom File Attributes can be used when more granular control is needed than can be achieved using the standard file attributes options (File Name, Path, File Type, File Size, Modified Time, Create Time, Metadata Change Time, Access Time). User Attributes use key value pairs to tag files with additional identifying criteria which SmartPools can then use to apply File Pool policies. While SmartPools has no utility to set file attributes this can be done easily by using the "find" command.

Custom File Attributes are generally used to designate ownership or create project affinities. For example, a biosciences user expecting to access a large number of genomic sequencing files as soon as personnel arrive at the lab in the morning might use Custom File Attributes to ensure these are migrated to the fastest available storage.

Once set, Custom File Attributes are leveraged by SmartPools just as File Name, File Type or any other file attribute to specify location, protection and performance access for a matching group of files. Unlike other SmartPools File Pool policies, Custom File Attributes can onll be set from the command line, not the UI.Anatomy of a SmartPools Job

When a SmartPools job runs, SmartPools examines all file attributes and checks them against the list of SmartPools policies. To maximize efficiency, SmartPools' examination step is a LIN walk (see figure 11) not a directory walk, meaning it is both highly parallelizable and efficient. A SmartPools LIN tree scan breaks up the LIN into ranges for each node to work on in parallel. Each node can then dedicate a single or multiple threads to execute the scan on their assigned range. A LIN tree walk also ensures each file is opened only once which is much more efficient when compared to a directory walk where hard links and other constructs can lead to single threading, multiple opens, etc.

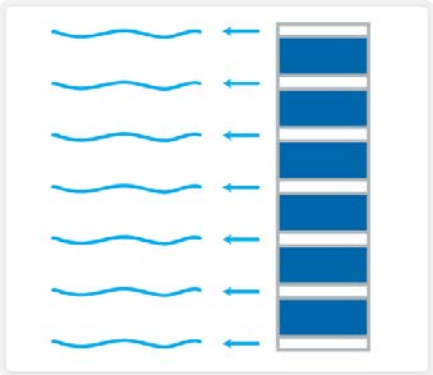

Figure 11. LIN Tree walk – LIN space is completely known before work begins so all work is segmented and distributed evenly among threads for faster job completetion

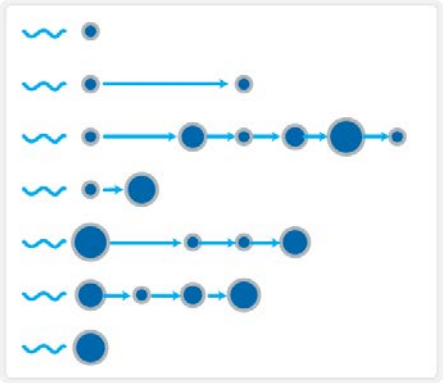

Figure 12. Directory Tree walk – Because tree depth and width is unpredictable, some may take much longer than others to complete

When it finds a match between a file and a policy, SmartPools stops processing policies for that file, as the first policy match is the one that determines what will happen to that file. Next, SmartPools checks the file's current settings against those the policy would assign to identify those which do not match. Once SmartPools has the complete list of settings that need to apply to that file, it sets them all simultaneously, and moves to restripe that file to reflect any and all changes to Disk Pool, protection, SmartCache use, layout, etc.

To test how a new policy will effect file disposition, a SmartPools job can be run on a single directory or group of directories. The job can be run live to actually make the policy changes, or non-op[8] to just calculate and display the end state of the files in the affected directory or directories. This means the end state can be simulated, showing how each file would be affected by the set of File Pool Policies in place. Running a SmartPools job against a directory or group of directories is a command line option only. For example:

To implement policies:

```
isi smartpools apply -v
```

To calculate and show output only without making changes:

```
isi smartpools apply -nv
```

When this approach is used, SmartPools uses a directory tree walk versus a LIN walk because the boundary of the operation has been specified at the directory level.

Data Location

For performance, charge back, ownership or security purposes it is sometimes important to know exactly with 100% certainty where a specific file or group of files is on disk at any given time. While any file in a SmartPools environment typically exists entirely in one Storage Pool, there are exceptions when a single file may be split (usually only on a temporary basis) across two or more Disk Pools at one time.

[8] Non-op = non operational; the job runs without actually implementing any changes just to show what the end stats of running the job *would* be.

The file system explorer provides a detailed view of where SmartPools-managed data is at any time by both the actual Disk Pool location and the File Pool policy-dictated location (where that file will move after a SmartPools job next runs).

When data is written to the cluster, SmartPools writes it to a single Disk Pool only. This means that, in almost all cases, a file exists in its entirety within a disk pool, and not across disk pools. Exceptions to this are discussed below. SmartPools determines which pool to write to based on one of two situations: If a file matches a File Pool policy based on directory path, that file will be written into the Disk Pool dictated by the File Pool policy immediately. If that file matches File Pool policy which is based on any other criteria besides path name, SmartPools will write that file to the Disk Pool with the most available capacity. If the File Pool policy that a file matches dictates it be placed on a different tier than the highest capacity Disk Pool, it will be moved when the next scheduled SmartPools job runs.

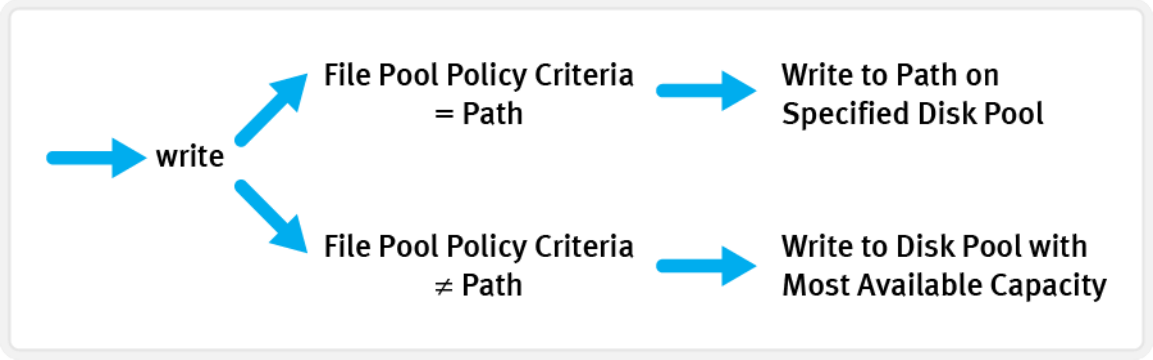

Figure 13. File Pools and Disk Pools

SmartPools generally allow a file to be only in one Disk Pool. A file may temporarily span several Disk Pools in some situations. When a file Pool policy dictates a file move from one Disk Pool to another, that file will exist partially on the source Disk Pool and partially on the Destination Disk Pool until the move is complete. If the Disk Pool configuration is changed (for example, when splitting a disk Pool into two Disk Pools) a file may be split across those two new pools until the next scheduled SmartPools job runs. If a Disk Pool fills up and data spills over to another Disk Pool so the cluster can continue accepting writes, a file may be split over the intended Disk Pool and the default Spillover Disk Pool. The last circumstance under which a file may span more than One Disk Pool is for typical restriping activities like cross-Disk Pool rebalances or rebuilds.

Performance with SmartPools

Because the main goals of storage tiering are to reduce data storage costs without creating data protection or access problems, it is important to address performance considerations.

There are several areas in which tiering can have a positive or negative performance impact on performance. It is important to consider each, and how SmartPools behaves in each situation.

Using SmartPools to Improve Performance

SmartPools can be used to improve performance in several ways:

- Location-based Performance
- Performance Settings
- SSD Strategies

Location-based performance is simple: it refers to the fact that with File Pool Policies, data can be placed on the media (SDD, SAS, SATA) which is most appropriate for the access performance required. A more advanced implementation of location-based performance is performance isolation. Using SmartPools, a Disk Pool can be isolated from all but the highest performance data, and File Pool policies can be used to direct all but the most critical data away from this Disk Pool. This approach is sometimes used to isolate just a few nodes of a certain type out of the cluster for intense work. Because Disk Pools are easily configurable, a larger Disk Pool can even be split, a resulting Disk Pool isolated and used to meet a temporary requirement, and then reconfigured back into a larger Disk Pool.

For example, in Figure 14 below, the administrators of this cluster are meeting a temporary need to provide higher performance support for a specific application for a limited period of time. They have split their highest performance tier and set policies to migrate all data not related to their project to other Disk Pools. The servers running their critical application have direct access to the isolated Disk Pool. A default policy has been set to ensure all other data in the environment is not placed on the isolated Disk Pool. In this way, the newly-created Disk Pool is completely available to the critical application.

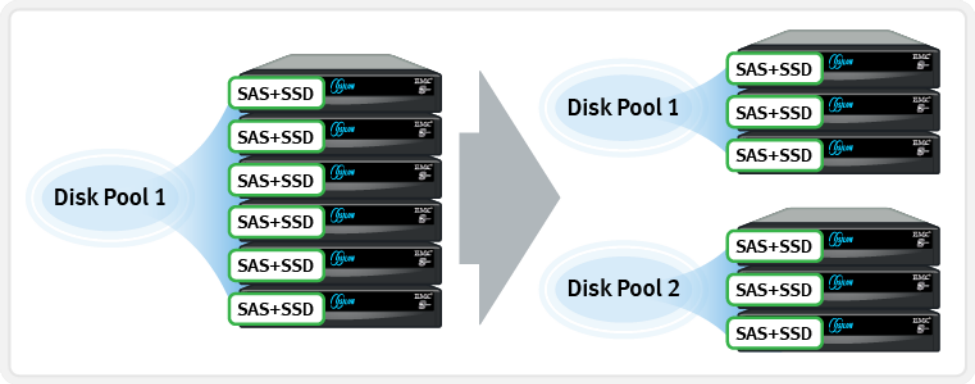

Figure 14. Disk Pool Example

Data Access Settings

At the File Pool (or even the single file) level, Data Access Settings can be configured to optimize data access for the type of application accessing it. Data can be optimized for Concurrent, Streaming or Random access. Each one of these settings changes how data is laid out on disk and how it is cached.

Data Access Setting	Description	On Disk Layout	Caching
Concurrency	Optimizes for current load on the cluster, featuring many simultaneous clients. This setting provides the best behavior for mixed workloads.	Stripes data across the minimum number required to achieve the data protection setting configured for the file[9].	Moderate prefetching
Streaming	Optimizes for high-speed streaming of a single file, for example to enable very fast reading with a single client.	Stripes data across a minimum of 15 drives.	Aggressive prefetching
Random	Optimizes for unpredictable access to the file, by adjusting striping and disabling the use of any prefetch cache. This is the default setting for iSCSI logical units.	Stripes data across the minimum number required to achieve the data protection setting configured for the file.	No (very little) prefetching

Table 3. SmartPools Data Access Settings

As the settings indicate, the Random access pattern turns off prefetching to avoid wasted disk accesses random access and works best for small files (‹ 128KB) and large files with random small block accesses. Streaming access works best for medium to large files with sequential reads. This access pattern uses aggressive prefetching to improve overall read throughput, and on disk layout spreads the file across a large number of disks to optimize access. Concurrent (the default setting for all file data) access is the middle ground with moderate prefetching. Use this for file sets with a mix of both random and sequential access.

Leveraging SSDs for Metadata & Data Acceleration

Adding SSDs within a Disk Pool can boost performance significantly for many workflows. In the Isilon architecture SSDs can be used to accelerate performance across the entire cluster using SSD Strategies for data or metadata acceleration.

There are several SSD Strategies to choose from: Metadata Acceleration, Avoid SSDs, Data on SSDs and Global Namespace Acceleration. The default setting, for Pools where SSDs are available, is 'Metadata Acceleration'.

Metadata Acceleration: Creates a mirror backup of file metadata on an SSD and writes the rest of the metadata plus all user data on HDDs. This approach dramatically reduces latency of metadata read requests. Depending on the global namespace acceleration setting, the SSD mirror may be an extra mirror in addition to the number required to satisfy the protection level. The SSD Mirror is to SSDs WITHIN the same

[9] See 'Data Protection in OneFS' above

pool as the data to which the metadata corresponds. If Global Namespace Acceleration (GNA) is not in use, the metadata of any file that migrates to a pool with no SSDs will reside on spinning media.

Global Namespace Acceleration (GNA) : With Global Namespace Acceleration (GNA) a copy of all metadata in the cluster is kept on SSD. With this option even the metadata associated with data on Storage Pools that have no SSDs is stored on the SSDs in nodes in different Storage Pools. File Pools policies which specify an "Avoid SSD" strategy can be used in combination with GNA so that SSD capacity is reserved to accelerate the metadata access for the most suitable or highest priority data.

Avoid SSDs: Never uses SSDs; writes all associated file data and metadata to spinning media only. This setting is used for data that cannot benefit from metadata acceleration or which is lower priority – ensuring SSD availability for higher priority data.

Data on SSDs: Similar to metadata acceleration, but also writes one copy of the file's user data (if mirrored) or all of the data (if not mirrored) on SSDs. Like GNA, Data on SSDs lowers metadata read times dramatically. Regardless of whether global namespace acceleration is enabled, any SSD blocks reside on the file's target pool if there is room. This SSD strategy does not result in the creation of additional mirrors beyond the normal protection level.

Minimizing the Performance Impact of Tiered Data

In addition to areas where SmartPools can be used to enhance performance, it is important to note that any tiering approach will have some cases where performance can suffer as a result of data location, tiering data movement activity.

Above we discussed data location and performance isolation as performance enhancement functions of SmartPools. As expected, if data is located on slower media it will be slower to access.

An interesting aspect to explore regarding data location on a scale-out NAS is that it is an architecture where data may be on nodes in one Disk Pool, while the network access point for the system(s) accessing that data may be via a very different nodes, with different front end IO capabilities, cache, etc. In this case the performance characteristics will depend on the specific workflow.

A good rule of thumb is that while SAS is going to be faster than SATA in most cases, and spindle counts can have a very important impact as well. The most important impact to performance in a SmartPools environment is the characteristics of the nodes in the Disk Pool the application is connecting to, rather than what type of media the data physically resides on. For example, assuming there is no bottleneck on the network side, a streaming application will see very little difference in performance between data on a SATA node with less CPU power versus a SAS node with more CPU power as long as its connection into the cluster is via a Disk Pool where the nodes have a great deal of cache. Similarly, for applications with Random access patterns, as long as the Disk Pool they connect into has sufficient CPU, in

most cases actual data location by spinning media type will not make a significant difference in performance.

Another important performance consideration in a tiered storage environment is the effect of data movement itself on overall system resources. The very action of moving data from one Disk Pool to another does use system resources. SmartPools mitigates the impact of data movement in multiple ways. First, architecturally, the job engine is very efficient and secondly through Impact Policies which are completely configurable by the end user to control how much system resource is allocated to data movement and when data movement takes place.

- **Paused**: Do not use cluster resources.
- **Low**: Use 10% or less of cluster resources.
- **Medium**: Use 30% or less of cluster resources.
- **High**: Use unlimited cluster resources.

Impact policy can be set once for SmartPools jobs, or controlled by a File Pool policy which can change the impact settings to match the typical peaks and lulls of the workload in a particular environment.

SmartPools Use Cases – Lowering Storage Costs without sacrificing performance or protection

Use Case 1: Storage Cost Reduction in Post Production

In a post production workflow each project deals with a very large number of very large files, can take a very long time to complete and must be archived as reference work for long periods. In the last 5 years alone, the amount of storage needed for post production has increased as much as 10 times and finished project sizes have doubled or tripled. Post production facilities are under increased pressure to provide faster turn times, richer effects and an ever greater number of post-release projects to market tie-ins and licensed products and derivatives.

Post production creates a massive amount of data as their main product. This can grow to 100s of Terabytes or even multiple Petabytes, and needs to be accessed very quickly by many people and processes while a project is active. Then it continues its rich commercial life within the company; being drawn on intermittently for follow on projects over long periods of time. This means that Post Production data is critical and timely, then not timely, then potentially timely again.

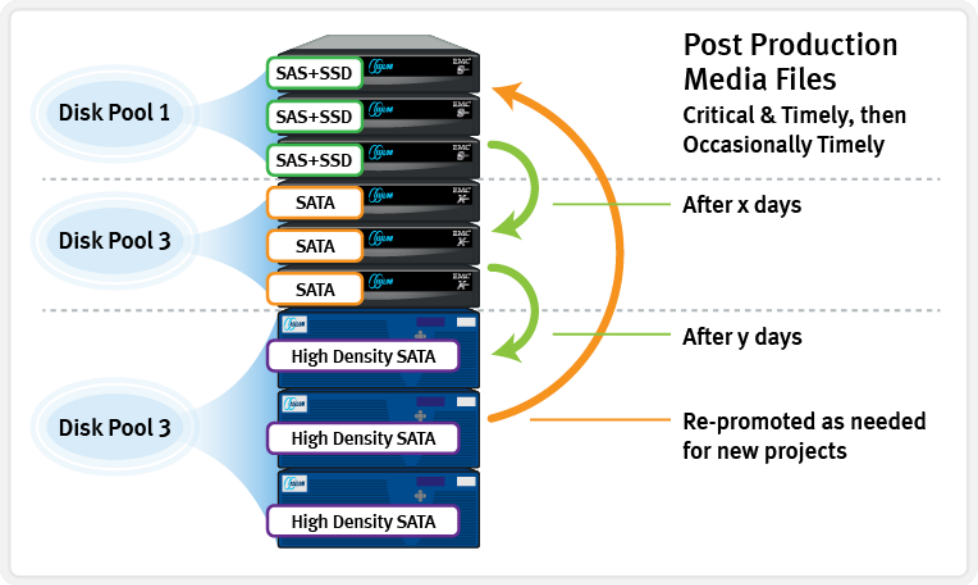

Figure 15. Media & Entertainment Post Production Use Case Example

In this situation, the increasingly popular approach is to have a two or three-tiered solution where working data is on high performance disk and all the rest is on lower performance storage, all of it set for streaming access and protected at a high level. This way, 60 to 95% of all data under management can be on less expensive disk, but it is all accessible at reasonable performance rates, and can be re-promoted to the fastest disk at any time and is completely protected for long periods of time.

Use Case 2: Data Protection in Manufacturing Design

An electronic design automation (EDA) workflow is usually an early step in the creation of a manufactured item such as an integrated circuit design or blueprints for an automobile engine. This design data is both critical and timely. The company is counting on the revenue from this product, and the workflow is high priority so multiple engineers from all over the world may be working on the same project, and all need to be able to get to the data quickly.

What about the data for previous designs? It is used as reference for new designs, for creating and defending the company's intellectual property portfolio and is essential in defending the company against product defect litigation. So it is definitely critical. Moreover, it can remain critical for many years, even decades. But it is not timely – no one has a need to access older designs at high performance speeds. All of the instances in which historical design data is referenced are not time critical enough to change the type of disk they are stored on, though an argument could be made against deep archive in the case of legal discovery timeframes. Therefore, older designs are critical, but not timely.

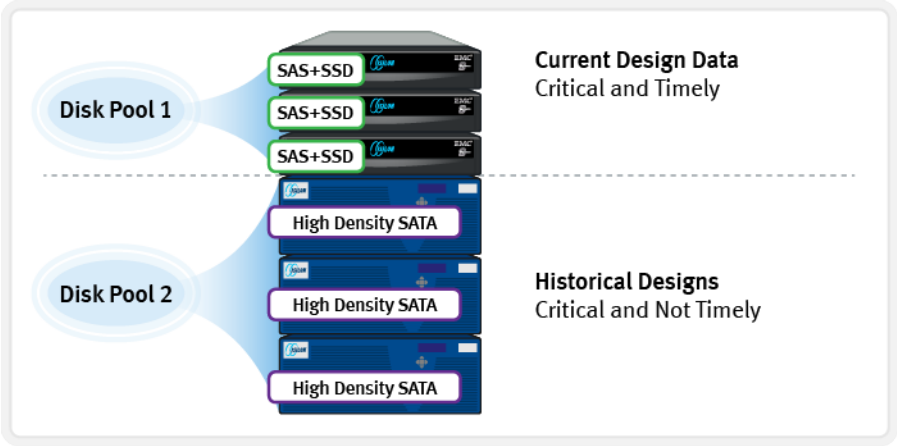

Figure 16. Data Protection in Manufacturing Design Use Case Example

These engineering design data and historical design data examples are valuable because they illustrate the need for data protection for two sets of data that exist side by side in the same infrastructure, one of which is timely and critical, the other critical and not timely. The need is clearly to have a system that can serve up data quickly or less expensively, but protect both data types equally.

Using SmartPools, this could be done most simply with a two-disk-pool architecture with high performance SAS or SATA (S or X Series) systems for the performance tier and high-capacity SATA (NL Series) as the less expensive tier. One File Pool policy would restrict Historical Design data to the high-capacity tier, protecting it at a high level (for example 3:1), while another would restrict current design data to the fastest tier at the same protection level.

Use Case 3: Performance with Global Namespace Acceleration

Seismic data used for oil and gas exploration activities can be huge with some companies keeping hundreds of terabytes, or even Petabytes of data online for faster analysis and more accurate drilling predictions. This data may be arranged in tens of thousands of files across thousands of directories. Less than 30% of those files may be used in any given month, but administrators constantly need to locate files on a daily basis for pre- and post-stack seismic workflows and project planning. Just getting a file system to return a list of its contents (a metadata operation) with this number of files and directories can take 5 or more minutes – unacceptable for a task that is repeated multiple times per day. Seismic data is the core of exploration and therefore it is critical, but for rarely-accessed files the data may not be timely, but the metadata is. In summary, seismic data is critical and its metadata is timely; its data may be timely or not timely

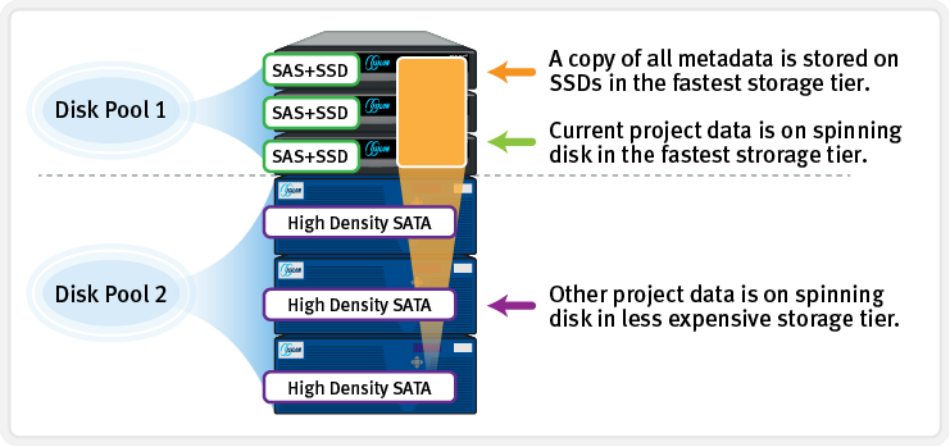

Figure 17. Global Namespace Acceleration Use Case Example

So, faster spinning media for the majority of the data is not necessarily needed if there is a way to speed up just the metadata access.

In this case, Global Namespace Acceleration is the logical approach. All of the metadata is stored on SSD, but data may be relegated to faster or slower disk based on its relative timeliness. The result is the ability to instantly locate any data, while still being able to cost reduce the majority of the data on the system. Because metadata is accelerated, there are performance benefits to many other activities as well including backups, migration and replication.

Use Case 4: Investment Protection for Financial Market Data

Financial market data is collected by an investment firm from multiple sources globally 24 hours a day. This real time data needs to be captured and stored, and is then referenced for multiple analysis processes and products the company uses to predict micro and macro market trends and support their purchase recommendations.

The data capture plus analysis results can generate terabytes of data per day. Real-time capture is free – if the real time feed is missed, the data must be purchased from a service, so performance and reliability are crucial. The majority of analysis is performed when the data is less than 48 hours old, but the data is kept "forever" so that less important but still very useful long term analysis and review of model accuracy over time can be done in any review timeframe.

Because this is a demanding production environment where data life is very long, there is a strong preference for systems that can live a long time and keep up with increases in requirements for capabilities and performance while minimizing costs.

In this case, they are likely to deploy a three tier SmartPools cluster where new data is acquired on mid-performance spinning media, analysis is run on high performance spinning media – possibly with SSDs – and older data that is used only intermittently would be moved to slower, less expensive disk.

Their original equipment stays online for many years – just moving down the hierarchy as completely capitalized capacity – while new drives and new nodes with better price and/or performance characteristics are added above them. With this investment protection approach, the company also avoids the pain of frequent forklift upgrades and the associated disruption to their workflow.

Conclusion

Traditional storage tiering implementations to date have been expensive, technically or administratively risky, or have simply not been able to align data value with accessibility, protection and performance needs.

SmartPools unique integration with the industry's leading Scale-Out NAS architecture delivers on the promise of storage tiering with significant storage cost savings without sacrificing performance or data protection.

With its simple, powerful interface, flexible options and intelligent default settings, SmartPools is easy to install and manage, and scales from terabytes to petabytes. Scalability to petabytes and the ability to add new capacity and new technologies while retaining older capacity in the same system means strong investment protection. Integration with OneFS core functions eliminates data migration risks and gives the user control over what system resources are allocated to data movement.

To learn more about SmartPools and other EMC Isilon products please see http://www.isilon.com/smartpools.

About EMC Isilon

As the global leader in scale-out storage, EMC Isilon delivers powerful yet simple solutions for enterprises that want to manage their data, not their storage. Isilon products are simple to install, manage and scale, at any size. And, unlike traditional enterprise storage, Isilon stays simple no matter how much storage is added, how much performance is required or how business needs change in the future. Information about Isilon can be found at http://www.isilon.com.

© 2012 Isilon Systems LLC. All rights reserved. Isilon, Isilon Systems, OneFS, and SyncIQ are registered trademarks of Isilon Systems LLC. Isilon IQ, SmartConnect, SnapshotIQ, TrueScale, Autobalance, FlexProtect, SmartCache, SmartPools, InsightIQ, "SIMPLE IS SMART," and the Isilon logo are trademarks of Isilon. Other product and company names mentioned are the trademarks of their respective owners.

U.S. Patent Numbers 7,146,524; 7,346,720; 7,386,675. Other patents pending.